

Karpathy LLM Wiki

本地知识库实战指南

用 Claude Code + Obsidian + LM Studio + Claudian + Web Clipper, 在自己的电脑上还原「左边知识库, 右边 AI 助手」 workflow

作者: 阿众

自媒体账号: @我是阿众, 全网同名

2026 年 5 月

视频配套电子资料

目 录

第1章 文档定位与读者预期

- 1.1 这是一份什么样的文档
- 1.2 这份文档适合谁读
- 1.3 读完能拿到的四样东西

第2章 开篇：为什么是 Claude Code，而不是别的工具

第3章 先讲清楚：Karpathy 的 LLM Wiki 方法论到底是什么

- 3.1 传统 RAG 的问题
- 3.2 LLM Wiki 的核心转变：从「检索」到「编译」
- 3.3 三层架构
- 3.4 三个核心操作
- 3.5 为什么这套玩法能成立

第4章 环境准备：先把工具装齐

- 4.1 Claude Code 安装
- 4.2 Obsidian 安装

第5章 给 Claude Code 配个模型：两种模式任选其一

- 5.A 模式 A：本地离线模型（LM Studio）
- 5.B 模式 B：第三方在线 API
- 5.X 两种模式一句话对比

第6章 安装 Claudian：把 Claude Code 嵌进 Obsidian

- 6.1 Claudian 是什么
- 6.2 系统要求
- 6.3 安装步骤
- 6.4 启用后看到的样子

6.5 一个小提示

第7章 把 Obsidian Web Clipper 调教好

7.1 安装

7.2 第一步配置：把 Vault 加进来

7.3 设置默认保存到 raw 文件夹（关键配置）

7.4 为不同类型资料做不同模板（进阶）

7.5 让图片也「落到本地」（很重要）

7.6 一个 LLM 处理图片的小常识

第8章 正式搭建知识库

8.1 创建空 Vault + 投喂 Karpathy 原文

8.2 AI 会自动建出来的目录结构

8.3 CLAUDE.md：AI 给自己写的员工手册

8.4 两个特殊文件：index.md 和 log.md

第9章 Ingest 操作：把资料「编译」进 wiki

9.1 怎么往 raw 里放资料

9.2 一条 ingest 指令背后发生了什么

9.3 Karpathy 自己的工作偏好

第10章 Query 操作：基于 wiki 提问

10.1 一次典型的查询是什么样的

10.2 为什么这套玩法彻底秒杀传统 RAG

10.3 复利效应：好答案要「回灌」成 wiki 页

10.4 输出格式不止于文字

第11章 Lint 操作：定期给 wiki 做体检

11.1 为什么需要 lint

11.2 让 AI 检查的清单

11.3 lint 的副产品

11.4 lint 的频率

第12章 进阶玩法（可选）

12.1 用 Obsidian 的 Graph View 看知识库的形状

12.2 给 wiki 配一个本地搜索引擎 qmd

12.3 用 git 管理整个 wiki

12.4 用 Marp 自动出 PPT

12.5 用 Dataview 做动态看板

第13章 典型应用场景

13.1 个人

13.2 研究

13.3 读书

13.4 公司 / 团队

13.5 其他

第14章 收尾

第1章 文档定位与读者预期

1.1 这是一份什么样的文档

这是一份**手把手的实战教程**，不是理论介绍，也不是综述。读完之后你应该能在自己电脑上跑通一整套系统，并且明白每一步背后的设计原理。

为什么强调原理？因为这套方法论有趣的地方不在于工具组合本身（工具一直在换），而在于它对「**LLM 和文档如何配合**」这件事做出的**全新假设**。你理解了底层逻辑，以后哪怕工具全换一遍，你也能自己重新搭。

1.2 这份文档适合谁读

- 对 AI 有基本认知，但不一定会编程的普通人。
- 平时有整理资料、做笔记、追研究主题、读专业书的需求。
- 用过 NotebookLM、ChatGPT 上传文件这类「传统 RAG」方案，但感觉始终差点意思的人。
- 想搭一套完全属于自己、长期沉淀的知识库的人。

1.3 读完能拿到的四样东西

1. 理解 Karpathy 提出的 LLM Wiki 方法论，知道它和传统 RAG 的本质区别在哪儿。
2. 在自己电脑上跑通一套 Claude Code + Obsidian 的完整环境。
3. 两种模型接入模式任选其一：要么走 LM Studio 本地离线，要么走第三方在线 API。两套都有完整的 settings.json 模板，直接复制就能用。
4. 学会四个核心操作：初始化、Ingest（导入编译）、Query（查询）、Lint（巡检）。

第2章 开篇：为什么是 Claude Code，而不是别的工具

Claude Code 的名字里带「Code」，很多人会下意识觉得它只能用来写代码。这是个误解。

Claude Code 的本质是一个**能读写本地文件、能执行命令、能多步行动**的 AI 助手。它的工作目录可以是你任何一个文件夹。这个文件夹里可以是代码，也可以是 Markdown 笔记、PDF 资料、CSV 数据。对它来说没区别。

事实上，对绝大多数普通用户而言，**让 Claude Code 帮你管理笔记和资料，比让它写代码更有价值**。因为：

- 写代码是程序员的需求，普通人没有。
- 但「整理资料、做笔记、查找历史信息」是所有知识工作者都有的需求。

前段时间，OpenAI 前研究员、Tesla 前 AI 总监、深度学习圈最受尊重的研究者之一 Andrej Karpathy，在 GitHub 上发布了一个 gist 文件，叫 `llm-wiki.md`。短短两千多字，没有代码，只描述了一种**用 LLM 来维护个人知识库的方法论**。这个 gist 在国内外 AI 圈儿很快火了起来，Star 数迅速突破 5000，社区里冒出了几十个开源实现。

这套方法论最妙的地方在于：它不依赖任何特定的工具，原文就一句话——「请把这份文件复制粘贴给你自己的 LLM Agent（比如 Claude Code、OpenAI Codex、OpenCode 等等），剩下的让你的 Agent 帮你搭」。

本文要做的事情，就是**基于这套方法论，给出一份具体的、可以照着操作的实战指南**。我们要还原的目标状态，是 Karpathy 本人在 gist 里描述的那个画面：

“「我会让 LLM Agent 开在一边，Obsidian 开在另一边。LLM 根据我们的对话进行编辑，我在旁边实时浏览结果——跟随链接、查看图谱视图、阅读更新后的页面。Obsidian 是 IDE，LLM 是程序员，wiki 是 codebase。」

第3章 先讲清楚：Karpathy 的 LLM Wiki 方法论到底是什么

在动手装软件之前，我们必须先把这套方法论彻底搞懂。否则你只是在机械地跟着步骤走，遇到问题就不知道怎么办。

3.1 传统 RAG 的问题

大多数人接触「LLM + 文档」的方式，是 RAG（Retrieval-Augmented Generation，检索增强生成）。具体表现就是：

1. 你上传一堆文件到某个工具（NotebookLM、ChatGPT、各种知识库产品）。
2. 你提一个问题。
3. 系统在你上传的文件里搜出相关片段。
4. LLM 基于这些片段拼一个答案出来。

这种模式能用，但有一个根本性的问题——**每次提问，LLM 都在从零开始重新发现知识。**

举个例子，你上传了 10 篇关于某个研究主题的论文。你问 LLM 一个需要综合所有论文才能回答的问题。它会去检索相关片段，拼出一个答案。然后你下一次再问类似的问题，它又得**重新检索一遍**，重新拼一遍。

它每次都在做同样的工作。它的「理解」是一次性的，每一轮对话结束，所有的综合、推理、关联，全部消失。

没有任何东西被积累下来。

这就是传统 RAG 的天花板。你的资料越多，每次查询的成本越高，但你得到的「智能」并没有累积。NotebookLM、ChatGPT File Uploads、市面上 90% 的 AI 知识库产品，全是这个套路。

3.2 LLM Wiki 的核心转变：从「检索」到「编译」

Karpathy 提出的方法论，把这件事整个翻了个面。

核心转变是：**LLM 不再是查询时的检索者，而是平时的维护者。**

具体来说：

- 当你往 raw 文件夹里丢一篇新资料的时候，LLM 不只是「记下来等着以后被搜」。
- 它会**立刻**读完这份资料，提炼关键信息，更新 wiki 里相关的实体页、概念页、综合分析页。
- 它会标注新数据和旧主张之间的矛盾。它会建立交叉引用。它会更新索引和日志。
- 知识被「编译一次，长期维护」，而不是「每次查询重新拼一次」。

这个改变带来的结果是什么？

当你下次提问的时候，**wiki 里已经写好了答案的素材**。交叉引用是现成的。矛盾已经被标记过。综合分析已经反映了你读过的所有东西。LLM 只需要去读 wiki，不需要去读原始资料。

Karpathy 用了一个非常贴切的比喻：

“Obsidian 是 IDE，LLM 是程序员，wiki 是 codebase。

你不会每次需要一个功能的时候，都让程序员从头重写一遍代码。代码会被一次次写好、维护、重构、积累下来。wiki 也一样。

3.3 三层架构

整套系统由三层组成。这三层是必须搞懂的核心模型。

第一层：Raw 层（原始资料层）

这一层放的是你收集的所有原始资料：网页文章、论文、PDF、播客转录稿、图片、数据文件。

关键规则：这一层是只读的，LLM 不能修改它。

为什么？因为这是你的「真理之源」（source of truth）。如果 LLM 可以修改原始资料，那么经过几轮迭代之后，你就再也分不清「这是原文写的」还是「这是 LLM 改写过的」了。所有的可信度就垮了。

所以纪律必须严格：**原始资料神圣不可侵犯。LLM 永远只读，永远不写。**

第二层：Wiki 层

这一层是 LLM 自己生成和维护的，包含：

- 每篇资料的摘要页（sources/）

- 实体页（entities/，人物、组织、产品、项目）
- 概念页（concepts/，术语、理论、方法论）
- 综合分析页（analyses/，对比、综合、由提问触发的产出）
- 索引文件 `index.md`
- 日志文件 `log.md`

关键规则：这一层完全归 LLM 管。你读它，但不写它。你想修改某个 wiki 页面，你应该让 LLM 去改，而不是自己手动改。

为什么？因为 wiki 的价值在于它是**自治的**。如果你手动改了一个页面，你可能没注意到这个改动还连带影响了其他五个相关页面、十几个交叉引用。LLM 才有能力同时维护这种整体一致性。

第三层：Schema 层

Schema 层只有一个文件，叫 `CLAUDE.md`（在 Codex 里叫 `AGENTS.md`，在其他 Agent 里有其他名字）。

这个文件告诉 LLM：

- 这个 wiki 的目录结构是什么样的
- 每种页面应该按什么格式写
- Ingest / Query / Lint 三个操作应该怎么执行
- 有哪些命名约定、标签规则、链接规范

这一层是让 LLM 从「通用聊天机器人」转变成「有纪律的 wiki 维护员」的关键。没有 schema，LLM 每次的行为都是随机的；有了 schema，它的行为就被规范住了。

schema 是你和 LLM **共同演化**的。一开始可能很简单，用着用着发现哪里不对，就让 LLM 帮你改 schema。它会越来越贴合你的工作流。

3.4 三个核心操作

整个系统的日常运作，靠三个动词支撑。

Ingest（导入编译）

你把一份新资料丢进 raw 文件夹，告诉 LLM：「请处理一下。」

它会：

1. 读完原文
2. 与你讨论关键点

3. 在 wiki/sources/ 写一份摘要页
4. 更新 wiki/entities/ 和 wiki/concepts/ 中的相关页面（已有则更新，没有则新建）
5. 用双向链接把这些页面交叉连起来
6. 更新 wiki/index.md
7. 在 wiki/log.md 追加一条日志

一份资料的导入，可能会触动 wiki 里的 10–15 个页面。这就是「编译」的含义——原始资料被解构、重组、整合进了一个有机的网络。

Query（查询）

你基于 wiki 提问。LLM 会先读 `index.md`，找到相关页面，再去读这些页面，综合一个带引用的答案。

关键点：好的回答应该被「回灌」成新的 wiki 页面。

你做了一次深度提问，LLM 给出了一份精彩的综合分析。这份分析不该消失在聊天历史里。让 LLM 把它存到 wiki/analyses/ 下，作为一份新的页面。

这意味着：你每一次提问，本身也在让知识库变得更聪明。

Lint（巡检）

定期（比如每 ingest 10 篇资料后）让 LLM 给整个 wiki 做一次健康检查。它会找：

- 不同页面之间的事实矛盾
- 被新资料推翻的旧论断
- 孤立页面（没有任何入链）
- 概念被提到但没有独立页面
- 缺失的交叉引用
- 可以通过 web search 补全的数据缺口

它还会顺手提出「接下来值得研究的新问题」「还该补哪些资料源」。

这三个操作，Ingest 是输入，Query 是输出，Lint 是维护。整个系统就是在这三个动作之间循环。

3.5 为什么这套玩法能成立

我们最后再追问一句：为什么这套方法论是有效的？

Karpathy 在 gist 里给出了一个非常精彩的洞察：

“维护一个知识库的痛苦，不在阅读，不在思考，而在「记账」——更新交叉引用、保持摘要新鲜、记录新数据推翻了哪些旧主张、维持几十个页面之间的一致性。

人类放弃 wiki，是因为维护负担增长得比价值快。

但 LLM 不会觉得无聊，不会忘记更新一个交叉引用，可以一次改 15 个文件。**wiki 能持续存在，是因为维护成本接近零。**

这句话值得反复读。

Vannevar Bush 早在 1945 年就提出了「Memex」的设想：一个私人的、被精心维护的知识库，文档之间有「联想路径」相连。这个设想后来被誉为「现代互联网的精神源头」。

但 Memex 七十多年来一直没能真正实现。**不是因为技术不够，而是因为没人愿意做那个维护的人。**

LLM 解决的，正是 Bush 当年解决不了的那个问题：**到底由谁来做维护。**

第4章 环境准备：先把工具装齐

我们要安装的软件清单：

软件	作用	必装
Claude Code	AI 大脑	必装
Obsidian	本地笔记载体	必装
Claudian	Obsidian 插件，把 Claude Code 嵌进侧边栏	必装
Obsidian Web Clipper	浏览器插件，一键剪藏网页到 Obsidian	必装
LM Studio	本地大模型运行环境	可选，只有走「模式 A 本地离线」才需要

4.1 Claude Code 安装

Claude Code 是 Anthropic 官方出的命令行 AI 工具。它是这套系统的「大脑」。

前置条件：

- Node.js v18 或更高版本（npm 安装路径需要）
- 或者直接走原生安装器，零依赖

安装方式有两种，任选其一。

方式一：原生安装器（推荐，零依赖）

macOS / Linux：

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
winget install Anthropic.ClaudeCode
```

方式二：通过 npm 安装

```
npm install -g @anthropic-ai/claude-code
```

注意

不要用 `sudo npm install -g`，会引发权限问题和安全风险。如果遇到 `EACCES` 权限错误，应该改 npm 的全局目录到你拥有所有权的位置：

```
mkdir ~/.npm-global  
npm config set prefix '~/.npm-global'  
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc  
source ~/.bashrc  
npm install -g @anthropic-ai/claude-code
```

验证安装：

```
claude --version
```

看到版本号输出就是装好了。

注意

关于账号：默认情况下 Claude Code 需要一个 Anthropic 的 Pro、Max、Team、Enterprise 或 Console (API) 账号。**免费的 Claude.ai 账号不能用 Claude Code。**但是！我们这份教程的核心是让 Claude Code 连本地模型或者第三方 API，所以你完全可以**跳过 Anthropic 账号这一步**。具体怎么配，看第五章。

4.2 Obsidian 安装

Obsidian 是目前最火的 Markdown 笔记软件之一。它最大的特点是——所有笔记文件都存在你本地的硬盘上，是一堆裸的 `.md` 文件，可以用任何编辑器打开。

这个特性对知识库来说至关重要。它意味着：

- 你的数据永远是你的，不会因为某个 SaaS 厂商关停而消失
- LLM 可以直接读写这些文件，没有任何 API 隔阂
- 即使你以后不想用 Obsidian 了，数据可以无损迁移到任何 Markdown 工具

下载地址：<https://obsidian.md>

安装完打开，新建一个 Vault（仓库）。这个 Vault 就是你未来的知识库根目录。建议起一个清晰的名字，比如 `my-llm-wiki` 或 知识库。

提示

现在还不要急着往里面放东西。我们要先把整个工具链装完，再统一初始化。

第5章 给 Claude Code 配个模型：两种模式任选其一

这一章是整个教程里**最关键的分叉点**。

Claude Code 本身只是一个 CLI 工具，它需要连接一个模型才能干活。这个模型可以是：

- **模式 A：本地离线模型**，跑在你自己的电脑上，通过 LM Studio 启动
- **模式 B：第三方在线 API**，比如 Anthropic 官方、OpenRouter、智谱 GLM、阿里云百炼等

两种模式底层用的是同一套机制：Claude Code 的 `ANTHROPIC_BASE_URL` 环境变量可以指向任何 **Anthropic 兼容的端点**。所以无论你选哪条路，配置方式都很像。

怎么选？看这张表：

你的情况	推荐模式
处理公司机密、合同、个人病历	模式 A
完全离线，一个字符都不想出网	模式 A
有像样的显卡（≥ 24GB 显存）或大内存 Mac	模式 A
没显卡，懒得折腾本地部署	模式 B
追求满血大模型的效果	模式 B
资料不敏感，想立刻跑起来	模式 B

提示

你只需要选一种走完，不需要两种都做。下面 5.A 和 5.B 是平行的两条路。

5.A 模式 A：本地离线模型（LM Studio）

5.A.1 适用场景

- 处理私密数据（公司机密、个人健康记录、合同等）
- 完全离线工作的需求
- 不想任何字符离开你的电脑
- 已经有比较好的硬件（独立显卡、或 Mac M 系列大统一内存）

5.A.2 安装 LM Studio

LM Studio 是一个可以在本地运行开源大模型的桌面应用，跨平台。

下载地址：<https://lmstudio.ai/download>

支持 macOS、Windows、Linux。如果你是在远程服务器或者 VM 上，可以装它的命令行版本 `lms`：

```
curl -fsSL https://lmstudio.ai/install.sh | bash
```

5.A.3 选模型不要只看参数规模

这是大家最容易犯的错。

很多人选模型只看「30B」「70B」这种规模数字，觉得越大越好。其实对本地部署来说，真正决定能不能跑起来、跑得流畅不流畅的，是另外两个指标：

1. 量化后的模型文件体积
2. 目标上下文长度对应的 KV 缓存占用

5.A.4 让模型跑得动的「显存公式」

记住这个公式：

```
模型文件体积 + KV 缓存占用 + 运行冗余空间 < 你的可用显存
```

举一个具体的例子。

你想跑一个 30B Q4 量化的模型：

- 模型文件体积：约 18-20 GB
- 想要 128K 的上下文：KV 缓存 + 推理时的中间状态再吃 10 GB 以上

- 运行冗余（操作系统、其他进程占用的显存）：再留 2-3 GB

加总：30 GB 以上的可用显存才能流畅跑起来。

显存 vs 推荐模型规模 速查表（仅作参考，实际还看具体模型架构和量化方式）：

可用显存	推荐模型规模（Q4 量化，64K 上下文）
8 GB	4B-7B 级别
12 GB	7B-9B 级别
16 GB	12B-14B 级别
24 GB	22B-27B 级别
32 GB	30B-32B 级别（紧凑）
48 GB	30B-32B 级别（宽松，128K 上下文）
64 GB+	70B 级别

提示

Mac M 系列说明：Mac 用的是统一内存（unified memory），不分显存和内存。一台 64GB 内存的 M3 Max，能跑的模型规模和一台 64GB 显存的 N 卡机器差不多。

5.A.5 在 LM Studio 里下载模型

打开 LM Studio，左侧搜索框里直接输入模型名字，比如：

- `qwen` —— 阿里通义千问系列
- `deepseek` —— 深度求索系列
- `glm` —— 智谱清言系列
- `gpt-oss` —— OpenAI 开源系列

示例：我的笔记本是 48GB 显存，所以我下载的是 30B 的 `GLM-4.7-Flash`（具体模型 ID 取决于 LM Studio 列表里显示的名字）。

点击下载按钮，等待几分钟到几十分钟（取决于网速和模型大小）。

5.A.6 启动本地 Anthropic 兼容 API

模型下载完之后，做以下几步：

1. 切换到 LM Studio 左侧的 **Developer**（开发者）选项卡
2. 在模型选择器中加载你刚下载的模型
3. 打开右上角的 **Server** 开关

服务启动后，LM Studio 会在 `http://localhost:1234` 暴露一个本地 API。

提示

重点知识：从 LM Studio 0.4.1 版本开始，它原生支持 Anthropic 兼容的 `/v1/messages` 端点。这意味着 Claude Code 可以直接连 LM Studio，不需要任何中转层（比如 LiteLLM 之类的代理）。这是这套方案能优雅工作的核心前提。

5.A.7 让 Claude Code 连上 LM Studio

找到 Claude Code 的配置文件：

- **Windows：** `C:\Users\<你的用户名>\.claude\settings.json`
- **macOS / Linux：** `~/.claude/settings.json`

如果文件不存在，自己创建一个。

需要配置的字段：

- `ANTHROPIC_BASE_URL`：填本地服务地址 `http://localhost:1234`
- `ANTHROPIC_AUTH_TOKEN`：本地服务不校验，随便填一个占位符
- 三个模型字段（Opus / Sonnet / Haiku）：全部指向你下载的模型 ID

完整 settings.json 模板（直接复制粘贴）：

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:1234",
    "ANTHROPIC_AUTH_TOKEN": "lmstudio",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "你的模型ID",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "你的模型ID",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "你的模型ID"
  }
}
```

注意

怎么知道模型 ID 是什么？ 在终端跑：

```
curl http://localhost:1234/v1/models
```

会返回 LM Studio 当前加载的模型列表，里面的 `id` 字段就是你要填的值。

举个完整例子，假设你下的是 `zai-org/glm-4.7-flash`：

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:1234",
    "ANTHROPIC_AUTH_TOKEN": "lmstudio",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "zai-org/glm-4.7-flash",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "zai-org/glm-4.7-flash",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "zai-org/glm-4.7-flash"
  }
}
```

5.A.8 上下文设置建议

LM Studio 官方推荐起步至少 **25K tokens** 的上下文。Claude Code 本身就很吃上下文，因为它要塞进系统提示、工具定义、对话历史、文件读取结果等等。

如果你的显存允许，开到 **64K** 甚至 **128K** 体验会好很多。设置位置在 LM Studio 加载模型时的 Context Length 选项。

5.A.9 验证连接

保存 settings.json 后，重新打开一个终端，输入：

```
claude
```

如果 Claude Code 的交互界面正常弹出，并且你能跟它对话，说明对接成功了。

5.A.10 常见坑

报错 / 现象	原因	解决
Connection refused	LM Studio Server 没开	回到 Developer 标签页打开 Server
404 Model not found	模型 ID 拼错了	<code>curl http://localhost:1234/v1/models</code> 查一下准确 ID
spawn claude ENOENT	Claude CLI 路径没找到	<code>which claude</code> 看路径，加到 PATH
模型回答很慢/卡死	显存不够，开始内存交换	减小上下文长度，或换更小的量化版本
工具调用失败	模型不支持 function calling	换一个明确支持 tool use 的模型

5.B 模式 B：第三方在线 API

5.B.1 适用场景

- 没有合适的本地硬件
- 不想折腾本地部署
- 追求和真 Claude / GPT 一样的效果
- 资料不太敏感，能接受走云端
- 想立刻跑起来开始用

5.B.2 主流的 Anthropic 兼容入口

由于 Anthropic 的 Messages API 已经成了一个事实标准，越来越多的厂商提供「Anthropic 兼容入口」。你可以用同一套 Claude Code 客户端，去连各种不同的后端。

服务商	入口 URL 示例	备注
Anthropic 官方	<code>https://api.anthropic.com</code>	最纯粹，直接用 Claude Opus / Sonnet / Haiku
OpenRouter	<code>https://openrouter.ai/api/v1</code>	一个 key 切换上百种模型
阿里云百炼 DashScope	<code>https://coding-intl.dashscope.aliyuncs.com/apps/anthropic</code>	国内访问稳定，可以用 Qwen 系列
智谱 Z.AI	<code>https://api.z.ai/api/anthropic</code>	GLM 系列的官方 Anthropic 兼容入口
Kimi / Moonshot	见官方文档	同理

共同点：所有这些服务商，都给你两样东西——一个 BASE_URL，一个 API Key。配置方式完全一样。

5.B.3 通用配置模板

打开 `~/.claude/settings.json`（Windows 是 `C:\Users\<用户名>\.claude\settings.json`），填入：

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "你在服务商那里申请到的 API Key",
    "ANTHROPIC_BASE_URL": "服务商给你的 Anthropic 兼容入口 URL",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "服务商给的模型名",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "服务商给的模型名",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "服务商给的模型名"
  }
}
```

5.B.4 几个常见服务商的具体填法

Anthropic 官方：

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "sk-ant-...",
    "ANTHROPIC_BASE_URL": "https://api.anthropic.com",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "claude-opus-4-7",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "claude-sonnet-4-6",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "claude-haiku-4-5-20251001"
  }
}
```

实际上，如果你用 Anthropic 官方账号，可以不填这些环境变量，直接在 **claude** CLI 里走 OAuth 登录就行。环境变量是给「绕过登录、走自定义端点」用的。

阿里云百炼（DashScope 国际版）：

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "你的 DashScope API Key",
    "ANTHROPIC_BASE_URL": "https://coding-intl.dashscope.aliyuncs.com/apps/anthropic",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "qwen3.5-plus",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "qwen3.5-plus",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "qwen3.5-plus"
  }
}
```

智谱 Z.AI：

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "你的 Z.AI API Key",
    "ANTHROPIC_BASE_URL": "https://api.z.ai/api/anthropic",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "glm-4.6",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "glm-4.6",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "glm-4.5-air"
  }
}
```

提示

具体的模型 ID 请以服务商最新文档为准，上面只是示例。

5.B.5 一个常见小贴士：避免登录弹窗

某些第三方服务商接入时，Claude Code 启动时会弹「Unable to connect to Anthropic services」错误，明明你已经配了第三方 BASE_URL。

解决办法是修改 `~/.claude.json`（注意，这个文件和 `~/.claude/settings.json` 不一样，是另一个文件，在你的 home 目录），添加：

```
{
  "hasCompletedOnboarding": true
}
```

如果这个文件不存在，新建一个就行。

5.B.6 验证连接

保存配置后，**重启你的终端**（不重启的话环境变量可能没生效），输入：

```
claude
```

看到对话界面正常出现并且能响应你的问题，就成功了。

5.X 两种模式一句话对比

维度	模式 A 本地离线	模式 B 在线 API
隐私	极佳，0 字符出网	取决于服务商
效果	受限于本地模型规模	满血大模型
成本	一次性硬件投入	按 token 计费
部署难度	需要装 LM Studio + 调显存	改 URL 和 Key 就行
联网依赖	完全不需要	必须联网
启动速度	模型加载慢，但之后秒响应	几乎即时
推荐人群	资料敏感、有像样硬件	想立刻跑起来的所有人

提示

一个组合用法：很多有经验的玩家会同时配两套，在不同任务里切换。比如：

- 处理公司内部资料 → 切到模式 A
- 做开放主题的研究 → 切到模式 B（用满血 Claude Opus）

切换方式很简单，写两份 settings.json，需要的时候改 ANTHROPIC_BASE_URL 这一行就行。

第6章 安装 Claudian：把 Claude Code 嵌进 Obsidian

到这一步，你的 Claude Code 已经能正常工作了。但它是个命令行工具，跟 Obsidian 是分开的。我们要的「左边 Obsidian、右边 Claude Code」体验，靠的是一个叫 **Claudian** 的 Obsidian 插件。

6.1 Claudian 是什么

Claudian 是一个**第三方 Obsidian 插件**，作者是 GitHub 用户 YishenTu。

它做的事情非常直观：**把 Claude Code 直接嵌进 Obsidian 的侧边栏。**

嵌进去之后：

- 你的 Obsidian 仓库（Vault）就成了 Claude Code 的工作目录
- Claude Code 能直接读你 Vault 里的文件、写文件、搜索、跑命令
- 一切操作的结果，你在 Obsidian 里实时可见

这正好完美对应 Karpathy 描述的工作流：**LLM 在改文件，你在旁边实时看结果。**

仓库地址：<https://github.com/YishenTu/claudian>

6.2 系统要求

- Obsidian v1.4.5 或更高版本
- 仅支持桌面端（macOS / Linux / Windows）
- 移动端目前不支持
- Claude Code CLI 已经装好（也就是第四章做完的事情）

6.3 安装步骤

由于 Claudian 不在 Obsidian 官方社区插件市场，我们要通过一个叫 BRAT 的辅助插件来装它。

第一步：装 BRAT 插件

BRAT 全称叫 Beta Reviewers Auto-update Tester，专门用来安装第三方 / 测试版插件。它本身在官方市场，可以直接搜到。

操作路径：

- 打开 Obsidian → Settings → Community plugins
- 如果你是第一次开社区插件，会有一个 “Turn on community plugins” 的提示，点开
- 点击 Browse，搜索 **BRAT**
- 点 Install，装完后点 Enable

第二步：用 BRAT 装 Claudian

- 进入 Settings → BRAT
- 点击 **Add Beta plugin**
- 在弹出框里填：<https://github.com/YishenTu/claudian>
- 点击 Add Plugin
- BRAT 会自动从 GitHub 拉最新版本

第三步：启用 Claudian

- 回到 Settings → Community plugins
- 在已安装插件列表里找到 Claudian
- 把它的开关打开

6.4 启用后看到的样子

在 Obsidian 左侧或右侧的 Ribbon（图标栏）上，会出现一个 Claudian 的图标。点击它，侧边栏会打开一个 Claude Code 对话框。

跟它打个招呼：「你好，请帮我看一下当前目录有哪些文件。」

如果它能正常响应，并且列出你 Vault 里的文件，说明整个链路通了。

恭喜你，到这一步，**你已经完成了整个工具链的搭建**。剩下的是知识库本身的初始化和日常工作流。

6.5 一个小提示

Claudian 的设置里有一个选项叫 `permissionMode`，默认是 `confirm`（每次执行操作前要你确认）。如果你觉得每次都确认太烦，可以改成 `yolo`（直接执行，不再问）。

注意

但建议你前期保持 `confirm` 模式，等熟悉了 Claude Code 的行为方式再放开。否则它可能在你不知情的情况下改了一堆文件，回滚起来很麻烦。

强烈建议：给你的 Vault 加上 git 版本控制，这样万一 AI 改坏了什么，你能一键回到上一个状态。

第7章 把 Obsidian Web Clipper 调教好

这一节单独拆出来，因为它直接决定了你后续往知识库里灌资料的体验是顺手还是难受。

Obsidian Web Clipper 是 Obsidian 官方出的浏览器插件，作用是**把网页一键变成 Markdown 文件存进你的 Vault**。

7.1 安装

各浏览器的安装入口：

- **Chrome / Edge / Brave / Arc / Vivaldi / Orion**（基于 Chromium 的浏览器）：
Chrome 应用商店搜 “Obsidian Web Clipper”
- **Firefox / Firefox Mobile**：Firefox Add-Ons 商店
- **Safari**（macOS / iOS / iPadOS）：App Store

安装完之后，建议把它固定到浏览器工具栏，剪藏时随手一点就行。

7.2 第一步配置：把 Vault 加进来

点击插件图标右上角的齿轮，进入 Settings → General。

找到 **Vaults** 这一栏，把你之前在 Obsidian 里建好的 Vault 名字填进去（注意，必须**和 Obsidian 里显示的名字完全一致**，包括大小写）。

这一步做完，剪藏时下拉框里才能选到这个 Vault。

提示

如果你有多个 Vault，可以全部加进来。Web Clipper 允许你在剪藏时选择目标 Vault。

7.3 设置默认保存到 raw 文件夹（关键配置）

注意

这是 Web Clipper 一个容易踩坑的设计点：保存位置不是全局设置，而是绑在「模板」上的。

也就是说，你不能简单地说「所有剪藏都保存到 raw 文件夹」，你必须通过模板来配置。

操作步骤：

1. 进入 Settings → Templates
2. 选中默认的 `Default` 模板（也可以新建一个）
3. 配置这几个关键字段：

字段	值	说明
Behavior	Create new note	保持默认，每次剪藏新建一个笔记
Note name	{{title}}	用文章标题做文件名；也可以加上日期等变量，如 {{date}} {{title}}
Note location	raw	核心配置： 这就是 Karpathy 方法论里的「收件箱」目录
Vault	选你之前在 General 里加进去的 Vault	默认使用哪个 Vault

保存。

从今往后，每次你点 Web Clipper 的图标剪藏，文章会自动落到 `<你的 Vault>/raw/` 文件夹下。

提示

可用的变量：除了 `{{title}}` 和 `{{date}}`，Web Clipper 还提供了很多变量可以用，比如：

- `{{author}}` —— 文章作者
- `{{published}}` —— 文章发布日期
- `{{description}}` —— meta 描述
- `{{url}}` —— 原始 URL

- `{{content}}` —— 文章正文

完整变量列表见 Obsidian 官方帮助文档: <https://help.obsidian.md/web-clipper/variables>

7.4 为不同类型资料做不同模板（进阶）

如果你的资料类型比较多样，可以为不同类型做专门的模板。

举个例子：

- **学术论文模板**：触发于 `arxiv.org/*`，保存到 `raw/papers/`，自动抽取作者、发表日期、摘要
- **视频文稿模板**：触发于 `youtube.com/*`，保存到 `raw/transcripts/`，抽取频道名、时长
- **新闻文章模板**：触发于其他网址，保存到 `raw/articles/`

操作：

1. 在 Settings → Templates 中点 `Add Template` 新建模板
2. 配置 Note name、Note location 等字段
3. 在 **Template triggers** 里设置 URL 模式（支持通配符）
4. 设置 properties（YAML frontmatter），自动写入元数据

剪裁时，Web Clipper 会根据 URL 自动匹配对应的模板。

7.5 让图片也「落到本地」（很重要）

Web Clipper 默认只保存 Markdown 文本，图片还是引用的远端 URL。这有两个问题：

1. 图床挂掉 / 网页改版，图就显示不出来了
2. 本地大模型没办法拿到网络图片

解决方案分两步。

第一步：在 Obsidian 里指定附件目录

- 打开 Obsidian → Settings → Files and links
- 找到 **Attachment folder path** 这一项
- 设为：`raw/assets/`

这样，所有下载下来的图片都会统一存到 `raw/assets/` 目录下。

第二步：绑定一个一键下载图片的快捷键

注意

版本要求：这个功能是 Obsidian **v1.8** 才引入的官方原生命令。所以确保你的 Obsidian 是新版本（v1.8 或更高）。

- 进入 Settings → Hotkeys
- 在搜索框里搜 `Download`
- 找到 `Download attachments for current file` 这条命令
- 给它绑一个顺手的快捷键
 - Windows / Linux：建议 `Ctrl + Shift + D`
 - macOS：建议 `Cmd + Shift + D`

使用流程：

1. 在浏览器上用 Web Clipper 剪一篇文章
2. 文章自动落到 `<你的 Vault>/raw/<文章标题>.md`
3. 在 Obsidian 里打开这篇文章
4. 按 `Ctrl + Shift + D`
5. 所有引用的外链图片都会被下载到 `raw/assets/`，文章里的图片链接自动改写成本地引用

整个过程一气呵成。

7.6 一个 LLM 处理图片的小常识

需要提一下：目前的 LLM 没办法在「读 Markdown 文本」的同时一次性「看见」内联图片。它们的工作方式是分两步：

1. 先读 Markdown 文本
2. 再用 view 工具看具体的图片文件

所以哪怕你把图片下载到本地了，LLM 也得分两步处理。这听起来有点笨，但实际用起来够用。

实操中你可以这样指挥 LLM：「请先读这篇文章的文字内容，然后挑几张关键的图看一下，再做总结。」

第8章 正式搭建知识库

环境装完了，模型配好了，Claudian 也嵌进 Obsidian 了，Web Clipper 也调好了。

到了正式搭建知识库的时刻。

8.1 创建空 Vault + 投喂 Karpathy 原文

如果你前面跟着做下来，应该已经有了一个空 Vault。如果还没有，现在新建一个，起名 `my-llm-wiki` 之类。

接下来做一件事：把 Karpathy 那篇 `llm-wiki.md` 原文直接拖进 Vault 根目录。

原文地址：<https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>

点击右上角 “Raw” 按钮，复制全文，新建一个 `llm-wiki.md` 文件粘进去；或者直接下载 zip 解压后拖进 Vault。

为什么要先把这份原文放进去？

因为我们要让 Claude Code 读这份原文，然后基于它的方法论来初始化知识库。这等于是让 AI 自己「读宪法」，然后按宪法办事。

打开 Claudian 侧边栏，跟它说：

“请深度阅读项目根目录下的 `llm-wiki.md` 这篇文章。这是 Andrej Karpathy 提出的 LLM Wiki 方法论。

请根据这套方法论，帮我初始化整个知识库的目录结构、`CLAUDE.md` schema 文件、`wiki/index.md` 和 `wiki/log.md` 等基础设施。

如果你按本教程的 8.3 节用现成的 `CLAUDE.md` 模板，可以略过让 AI 自己生成 `CLAUDE.md` 那一步，直接说「请只帮我创建目录结构和 `index.md`、`log.md`，`CLAUDE.md` 我自己来」。

8.2 AI 会自动建出来的目录结构

正常情况下，AI 会创建出类似这样的结构：

```
my-llm-wiki/
├── raw/                # 原始资料层（只读，LLM 不得修改）
│   ├── assets/         # 图片、附件等媒体文件
│   └── llm-wiki.md      # 你扔进去的 Karpathy 原文
│
├── wiki/               # Wiki 层（LLM 完全负责写作与维护）
│   ├── entities/       # 实体页：人物、组织、产品、项目
│   ├── concepts/       # 概念页：术语、理论、方法论
│   ├── sources/        # 来源摘要页：每个原始资料对应一个摘要
│   ├── analyses/       # 分析页：比较、综合、回答问题的产出
│   ├── index.md        # 内容目录（按类别列出所有 wiki 页）
│   └── log.md          # 活动日志（只追加，不修改历史）
│
└── CLAUDE.md           # Schema 与操作规范（员工操作手册）
```

每个目录的作用：

- **raw/**：你的「收件箱」。所有原始资料都丢在这里。LLM 只读不写。
- **raw/assets/**：图片附件目录。和我们在 7.5 节配置的 Attachment folder path 对应。
- **wiki/entities/**：实体页。人物、组织、产品、项目，一个实体一个页面。
- **wiki/concepts/**：概念页。术语、理论、方法论，一个概念一个页面。
- **wiki/sources/**：来源摘要页。每篇 raw 里的资料对应一个摘要页。
- **wiki/analyses/**：分析页。由你提问触发，或 lint 过程产出的综合分析。
- **wiki/index.md**：整个 wiki 的目录大纲。AI 查询时第一个读的就是它。
- **wiki/log.md**：操作日志，append-only。
- **CLAUDE.md**：Schema 文件。AI 给自己写的「员工操作手册」。

8.3 CLAUDE.md：AI 给自己写的员工手册

整个系统里最关键的一份文件，就是这个 `CLAUDE.md`。

它的作用是让 Claude Code 从一个「通用聊天机器人」变成一个「有纪律的 wiki 维护员」。它规定了：

- 目录结构是什么样的
- 每种类型的页面应该按什么格式写
- Ingest / Query / Lint 三个操作应该怎么执行
- 哪些东西可以修改，哪些不能
- 每次会话开始应该读哪些文件来对齐状态

可以把它理解成给 AI 看的「员工守则 + 工作流程文档 + 公司章程」。

注意

重要前置说明：下面这份 CLAUDE.md 里频繁出现的「LLM」「Claudian」字眼，指的就是你接入 Claudian 之后的 Claude Code。如果你用的是其他 Agent（Codex 等），把 LLM 理解为「你接入的那个 AI」就行。

提示

你有两条路线可以走：

路线一：让 AI 根据 Karpathy 原文自动生成 CLAUDE.md 适合首次跑通的人，让 AI 和你的工作流自然磨合。生成完之后再根据使用感受迭代。

路线二：直接用我准备好的成品模板（推荐） 我已经迭代过几个月，下面这份是相对成熟的版本。直接复制粘贴到你 Vault 根目录的 CLAUDE.md 文件里就能用，省去让 AI 自己摸索的那一步。

8.3.1 完整可复制的 CLAUDE.md 模板

注意

使用方式：把下面这段全部复制到 <你的 Vault>/CLAUDE.md 文件里。然后在 Claudian 里跟 AI 说一句「请阅读 CLAUDE.md 并根据它初始化目录结构和 wiki/index.md、wiki/log.md」即可。

```
# Claude Wiki Schema

这是本知识库的 **Schema 文件**，告诉 LLM（Claudian）如何维护这个 Wiki。每次新会话开始时，先读取此文件，再读取 `wiki/index.md` 和 `wiki/log.md`，然后再操作任何其他文件。

---

## 目录结构

...

/
|—— raw/                # 原始资料层（只读，LLM 不得修改）
|   |—— assets/          # 图片、附件等媒体文件
|   |—— *.md / *.pdf / ... # 文章、论文、笔记等原始文档
|
|—— wiki/                # Wiki 层（LLM 完全负责写作与维护）
|   |—— entities/        # 实体页：人物、组织、产品、项目
|   |—— concepts/        # 概念页：术语、理论、方法论
|   |—— sources/         # 来源摘要页：每个原始资料对应一个摘要
|   |—— analyses/        # 分析页：比较、综合、回答问题的产出
|   |—— index.md         # 内容目录（按类别列出所有 wiki 页）
|   |—— log.md           # 活动日志（只追加，不修改历史）
|
|—— CLAUDE.md            # 本文件：Schema 与操作规范
...

---

## 页面格式规范

### 所有 Wiki 页面的 YAML Frontmatter

```yaml

title: 页面标题
type: entity | concept | source | analysis
tags: [tag1, tag2]
created: YYYY-MM-DD
```

updated: YYYY-MM-DD

sources: 0 # 本页引用的原始资料数量

---

```

entities/ — 实体页

用于描述具体的人物、组织、产品、项目。

结构：

- **概述**：一段简介
- **关键信息**：重要属性列表
- **相关概念**：链接到 `concepts/` 中的相关页
- **来源**：链接到 `sources/` 中引用的来源摘要

concepts/ — 概念页

用于解释术语、理论、方法论。

结构：

- **定义**：精炼的定义
- **核心要点**：要点列表
- **与其他概念的关系**：链接相关概念
- **相关实体**：链接到 `entities/` 中涉及该概念的实体
- **来源**：链接到支撑该概念的来源摘要

sources/ — 来源摘要页

每个原始资料（`raw/` 中的文件）对应一个摘要页。

结构：

- **原始资料**：链接到 `raw/` 中的原文
- **摘要**：200-500 字的核心内容提炼
- **关键主张**：列出文章中的关键观点
- **更新的 Wiki 页面**：列出本次 ingest 更新了哪些页面
- **待深入探索**：本资料引发的问题或值得追查的线索

analyses/ — 分析页

由用户提问触发，或 lint 过程产出。

结构：

- ****问题****：触发此分析的问题
- ****答案/发现****：综合多个来源的分析
- ****引用****：列出引用的 wiki 页面
- ****延伸问题****：分析过程中产生的新问题

操作 workflow

Ingest（摄取新资料）

1. 读取 `wiki/index.md` 和 `wiki/log.md` 了解现状
2. 读取 `raw/` 中的新资料
3. 与用户讨论关键点
4. 在 `wiki/sources/` 创建摘要页
5. 更新 `wiki/entities/` 和 `wiki/concepts/` 中的相关页（有则更新，无则新建）
6. 更新 `wiki/index.md`
7. 在 `wiki/log.md` 末尾追加一条日志（格式见下）

Query（查询）

1. 读取 `wiki/index.md` 找到相关页面
2. 读取相关页面并综合答案
3. 如果答案值得保留，在 `wiki/analyses/` 创建分析页，并更新 index

Lint（健康检查）

检查：

- 有无矛盾的页面
- 有无孤立页面（无入链）
- 有无提到但缺少独立页面的重要概念
- 有无缺失的交叉链接
- 有无可被新来源补充的数据缺口

```
## index.md 格式

```markdown
Wiki 目录

最新更新: YYYY-MM-DD | 共 X 个页面 | 已摄取 X 个来源

实体 (entities/)
| 页面 | 简介 | 更新日期 |
|-----|-----|-----|
| [[wiki/entities/xxx]] | 一句话描述 | YYYY-MM-DD |

概念 (concepts/)
...

来源摘要 (sources/)
...

分析 (analyses/)
...
```

---

## log.md 格式

每条日志以 `## [YYYY-MM-DD] type | Title` 开头, 便于 grep 解析:

```
...
[2026-05-10] ingest | 文章标题
[2026-05-10] query | 提问内容摘要
[2026-05-10] lint | 健康检查
...
```

---

## 重要原则

1. **原始资料不可修改**: `raw/` 下的文件只读, LLM 只读不写。
```

2. ****Wiki 由 LLM 全权维护****：用户读，LLM 写。
3. ****日志只追加****：`log.md` 永远追加，不修改历史记录。
4. ****交叉链接是核心价值****：每次更新都要维护 Wiki 内的链接网络。
5. ****每次会话开始必读****：`CLAUDE.md` → `wiki/index.md` → `wiki/log.md`。

8.3.2 使用提示

把上面这段内容粘贴到 `<你的 Vault>/CLAUDE.md` 之后，注意三件事：

1. **文件名必须严格写成 `CLAUDE.md`**，全大写。Claude Code 会自动识别这个文件名，作为它在这个项目里的「上下文文件」。
2. **必须放在 Vault 根目录**，不能放进子文件夹。
3. **CLAUDE.md 是活的**。随着你使用，会发现某些地方不太顺手。直接让 AI 帮你改 CLAUDE.md，比如「请在 ingest 工作流的第 4 步前加一条：先和用户确认 source 页的 slug 命名」。让 schema 跟着你的工作流一起演化。

提示

第一次跑的建议：粘贴完 CLAUDE.md，跟 AI 说：

“请阅读 CLAUDE.md，然后根据里面定义的目录结构，创建 raw/、raw/assets/、wiki/、wiki/entities/、wiki/concepts/、wiki/sources/、wiki/analyses/ 这些目录，并初始化空的 wiki/index.md 和 wiki/log.md 文件。

它会一步步执行，几秒钟之内你的知识库骨架就搭好了。

8.4 两个特殊文件：index.md 和 log.md

这两文件在整个系统里扮演了非常微妙但关键的角色，单独说一下。

index.md：内容导向的目录

它列出 wiki 里所有页面，按类别组织，每条带一句话摘要。它的用途：

- **AI 查询时第一个读的文件**。它先读 index.md 找到相关页面，再去读这些页面。
- 你自己想浏览知识库时，也从这里开始。
- 在小规模（一两百个页面）下，**用 index.md 比用向量数据库 RAG 更优**——因为它已经是 AI 友好的人类摘要，不需要昂贵的相似度检索。

每次 ingest 完, AI 都应该更新 index.md。

log.md: 时间导向的日志

它按时间顺序记录每一次 ingest、query、lint。只追加, 不修改历史。

它的用途:

- 让你知道知识库的演化历程
- 让 AI 知道「最近发生了什么」, 避免重复劳动
- 用统一的前缀格式 (## [YYYY-MM-DD] type | title), 可以用 `grep` 等命令行工具快速搜索

Karpathy 在原文里推荐用 `grep` 来查最近几次操作:

```
grep "^## \[" wiki/log.md | tail -5
```

这条命令会给你最近 5 条日志条目。简单暴力但有效。

第9章 Ingest 操作：把资料「编译」进 wiki

环境装好了、knowledge base 骨架也搭好了。从这一章开始进入日常使用的循环。

9.1 怎么往 raw 里放资料

资料的来源主要有这么几种：

来源一：网页文章

按第七章配好 Web Clipper 之后，剪裁流程是：

1. 在浏览器里看到一篇好文章
2. 点 Web Clipper 图标
3. 检查一下默认模板和保存路径
4. 点 Add to Obsidian
5. 文章自动落到 `<Vault>/raw/<标题>.md`
6. 切回 Obsidian，打开这篇文章
7. 按 `Ctrl + Shift + D`（或你绑定的快捷键），一键把所有外链图片下载到 `raw/assets/`

来源二：PDF / 论文

直接把 PDF 文件拖进 raw 文件夹。Claude Code 能读 PDF。

来源三：手写笔记 / 转录稿

你录的会议纪要、和别人聊天的笔记、播客转录稿，都直接保存成 `.md` 文件丢进 raw。

来源四：已有的笔记仓库

如果你之前在别的工具有笔记（Notion、印象笔记等），可以批量导出成 Markdown，全部丢进 raw。让 AI 慢慢消化。

注意

再次强调：raw 是只读层。一旦资料进去了，**别再手动改它**。要修正信息，应该通过新增 source（比如新的资料、更新版本）的方式让 wiki 自动吸收。

9.2 一条 ingest 指令背后发生了什么

打开 Claudian 对话框，丢给 Claude Code 一句话：

“raw 里新增了一篇文章 xxx.md，请按 CLAUDE.md 里定义的 ingest 流程处理一下。”

它会自动执行下面这一连串动作：

步骤 1：先看现状 读取 wiki/index.md 和 wiki/log.md，了解当前 wiki 有哪些页面、最近做过什么操作。

步骤 2：读原文 打开 raw/xxx.md，把全文读一遍。

步骤 3：和你讨论（这一步取决于你的 CLAUDE.md 是否要求 LLM 主动确认）它可能会跟你说：「我读完了。这篇文章核心讲的是 ABC，主要涉及 X、Y、Z 几个概念。我准备这样处理：……请确认。」

步骤 4：写摘要页 在 wiki/sources/ 下创建一个 source 页，写好 200–500 字的摘要、关键主张、本次 ingest 影响了哪些页面、待深入探索的线索。

步骤 5：更新实体页和概念页 对文章里出现的每一个重要实体（人物、组织、产品）和概念（术语、理论），如果 wiki 里已经有对应页面，**就更新它**；如果没有，**就新建一个**。

步骤 6：建立交叉链接 所有页面之间用 Obsidian 的 [[双向链接]] 语法相互连接起来。这是整个 wiki 价值的核心——一个页面被引用得越多，说明它越是知识网络的「枢纽」。

步骤 7：更新 index.md 把新增或修改的页面，反映到 wiki/index.md 的目录里。

步骤 8：追加 log.md 在 wiki/log.md 末尾追加一条日志：

```
## [2026-05-27] ingest | 文章标题
- 创建: wiki/sources/xxx.md
- 更新: wiki/entities/yyy.md, wiki/concepts/zzz.md
- 新建: wiki/entities/aaa.md
- 待探索: bbb、ccc 这两个概念在文章里被提到但没展开
```

一篇资料的导入，可能会同时触动 wiki 里的 10–15 个页面。这就是「编译」的含义。

9.3 Karpathy 自己的工作偏好

Karpathy 在 gist 里分享了他自己用这套方法论的偏好，值得参考：

“我个人偏好一次只 ingest 一篇资料，全程在旁边看——我读摘要，检查更新，引导 LLM 强调什么。

但你也可以批量 ingest 很多资料、降低监督程度。这取决于你的风格。把你的工作流写进 schema 里，方便以后会话沿用。

我的建议是：

- 前 10 篇资料：一次一篇，全程在旁边盯着。看 AI 怎么处理，及时纠正它的方向。
- 第 10–50 篇：可以小批量，每批 2–3 篇，做完后整体审一次。
- 50 篇以后：schema 已经成熟，可以放手让它自动跑。

为什么前期要慢？因为前期是你和 AI 对齐工作流的关键期。AI 第一次处理某个领域、某种风格的资料时，往往会有一些不太对的判断，比如：

- 把不该独立成页的小细节单独建了页面
- 同一个概念被建成了两个页面（命名不一致）
- 链接方向反了（应该是 A 指向 B 的，建成了 B 指向 A）

前期及时纠正这些，让 AI 学到「我们这个 wiki 的偏好」。学到的东西，可以让它自己写进 CLAUDE.md 里，下次会话就会自动遵守。

第10章 Query 操作：基于 wiki 提问

10.1 一次典型的查询是什么样的

打开 Claudian，问一个问题：

“我前段时间读的那几篇关于 Karpathy 方法论的文章，核心论点和我之前读的几篇关于 RAG 的论文有哪些关联？请综合分析一下，并明确告诉我每个观点来自哪份原始资料。

观察 Claude Code 的动作：

1. 先打开 `wiki/index.md`，扫一眼整体目录，找到「Karpathy」「RAG」相关的页面
2. 进入 `wiki/concepts/` 下的 `LLM-Wiki.md`、`RAG.md` 等概念页
3. 进入 `wiki/sources/` 下相关的几篇 source 摘要
4. 可能还会去读 `wiki/analyses/` 下之前你做过的综合分析
5. 综合所有内容，给出一个结构化的回答
6. 每一个观点后面，明确标注引用来源

整个过程你能在 Claudian 里实时看到它在打开哪些文件。这种透明度比传统 RAG 黑盒强一万倍。

10.2 为什么这套玩法彻底秒杀传统 RAG

让我们对比一下：

| 维度 | 传统 RAG | LLM Wiki |
|---------|------------------|-------------------------|
| 每次查询的成本 | 全量向量检索 + 重排 + 拼接 | 读 index.md → 读几个具体页面 |
| 答案的连贯性 | 受限于检索到的片段，常常有断层 | wiki 本身已经做过综合，连贯自然 |
| 跨文档关联 | 几乎没有，靠 LLM 临时拼 | 已经在 ingest 阶段建好双向链接 |
| 矛盾标注 | 没人做，靠 LLM 临时发现 | 已经在 ingest / lint 阶段标记好 |
| 知识沉淀 | 0 沉淀，每次都从零开始 | 每次提问都让 wiki 变得更聪明 |
| 可解释性 | 黑盒，看不到检索过程 | 完全透明，能看到 AI 读了哪些文件 |

最关键的一点：传统 RAG 的智能上限，等于它的检索算法上限。

LLM Wiki 的智能上限，等于你 wiki 的累积深度——你用得越久，它越聪明。

10.3 复利效应：好答案要「回灌」成 wiki 页

这是这套方法论最妙、也最容易被忽视的一招。

当 AI 给你一个高质量的综合答案之后，别让它消失在聊天历史里。跟它说：

“
这个分析非常棒。请把它存到 wiki/analyses/ 下作为一份新的页面，并按 CLAUDE.md 里 analyses/ 的格式来写。记得在末尾列出引用的所有 wiki 页面，并更新 index.md。
”

它会：

- 1. 在 wiki/analyses/ 下新建一个分析页
- 2. 把答案按规范格式整理好
- 3. 列出引用的所有 wiki 页面
- 4. 列出延伸出的新问题（值得后续追查的线索）
- 5. 更新 index.md，在 analyses/ 那一栏加上这条新分析
- 6. log.md 追加一条 query 日志

这意味着，你的每一次提问，都在让知识库继续生长。

下次你再问类似的问题，AI 会先读到这份分析页，节省一大段重新综合的过程。你的知识库像生命体一样在长。

这是为什么 Karpathy 把这件事叫「complexion of knowledge」——知识的「肤色」会随着你的提问和回答，逐渐变深、变厚、变独特。

10.4 输出格式不止于文字

LLM Wiki 的回答可以采取多种形式：

- 一份 **Markdown 分析页**（最常用）
- 一张**对比表**（适合做横向比较）
- 一份 **Marp 幻灯片**（Obsidian 有 Marp 插件，可以直接出 PPT）
- 一张**图表**（matplotlib / mermaid，让 AI 写代码出图）
- 一张 **Canvas 关系图**（Obsidian 内置 Canvas 功能）

跟 AI 明确说你要什么格式就行。比如：

“请把上面的分析做成一份 5 页的 Marp 幻灯片，按「问题 → 三个核心观点 → 结论」的结构组织。

第11章 Lint 操作：定期给 wiki 做体检

11.1 为什么需要 lint

wiki 长大后会产生「熵」。具体表现：

- **矛盾**：A 页面说 X 是 5 亿用户，B 页面说 X 是 3 亿用户
- **过时**：去年的论断被今年的新数据推翻了
- **孤立**：某个页面没有任何入链，是个「断了线的风筝」
- **缺漏**：「Transformer」被反复提到，但没有自己独立的概念页
- **断链**：A 页面引用了 B 页面，但 B 页面已经被改名了
- **数据空缺**：某个实体的关键属性是空的（可以联网补全）

如果不管，wiki 会越来越脏，最后变成一个谁也不敢碰的烂摊子。

Lint 的作用，就是定期做一次「系统体检」。

11.2 让 AI 检查的清单

跟 AI 说：

“请按 CLAUDE.md 里 Lint 流程，对整个 wiki 做一次健康检查。

重点关注：

1. 不同页面之间的事实矛盾
2. 被新资料推翻的旧论断
3. 孤立页面（没有任何入链）
4. 被反复提到但没有独立页面的重要概念
5. 缺失的交叉引用
6. 关键属性空缺、可以通过 web search 补全的数据

请生成一份 lint 报告，列出发现的问题。不要直接修改，先把问题列给我看，我决定哪些要修。

注意

强烈建议「报告先行」：不要让 AI 在 lint 时直接动手改东西。先看报告，再决定哪些修、哪些不修。否则它可能改坏一些你认为「特意保留的不一致」。

11.3 lint 的副产品

除了发现问题，lint 还有几个有价值的副产品：

- **下一步研究方向**：AI 会顺手提出「你这块知识在 XYZ 角度有空白，可以找几篇相关资料补一下」
- **概念整合建议**：AI 可能发现「A 和 B 这两个概念页其实是同一回事，建议合并」
- **缺失的资料类型**：AI 会指出「你的 wiki 里关于 X 主题的资料全是技术性的，缺少商业视角的来源」

这些副产品**比修复问题本身更有价值**，因为它们在告诉你「下一步该读什么」。

11.4 lint 的频率

建议：

- **每 ingest 10 篇资料**，做一次轻量级 lint（只检查最近变动的部分）
- **每 ingest 50 篇资料**，做一次全量 lint（扫整个 wiki）
- **每隔几个月**，做一次「主题深度 lint」（选一个主题，让 AI 对这个主题的所有相关页面做深度梳理）

第12章 进阶玩法（可选）

到这里，主流程已经讲完了。下面是一些可选的进阶玩法，可以让这套系统更顺手。

12.1 用 Obsidian 的 Graph View 看知识库的形状

Obsidian 自带 Graph View 功能。打开它（左侧 Ribbon 上的图标），你会看到整个 wiki 以可视化的方式呈现。

它能帮你看出：

- 哪些页面是 **Hub**（节点很大、连线很多的页面）
- 哪些页面是**孤岛**（独立的小圆点）
- 知识结构是否**平衡**（某些区域过密 / 过疏）

定期看 Graph View，对你的 wiki 的整体形状会有更直观的感受。

12.2 给 wiki 配一个本地搜索引擎 qmd

小规模下（一两百个页面），index.md 就足够支撑查询。但 wiki 规模上来之后，你会想要一个更强的搜索引擎。

Karpathy 在 gist 里推荐了 **qmd** —— 一个本地的 Markdown 搜索引擎。

- 作者：Tobi Lütke（Shopify 创始人）
- 地址：<https://github.com/tobi/qmd>
- 特点：
 - 完全本地运行，零云依赖
 - 混合搜索（BM25 + 向量语义 + LLM 重排）
 - 同时支持 CLI 和 MCP server 两种使用方式

接入 Claude Code 后，AI 可以通过 MCP 协议直接调用 qmd 来做搜索，效率比一个个读文件高很多。

12.3 用 git 管理整个 wiki

整个 wiki 本质上就是一个 Markdown 文件夹。给它加上 git 之后，你免费获得了：

- **版本历史**：任何时候都能回到过去某个状态
- **分支**：可以开个分支做实验性的 wiki 重构，不影响主线
- **协作**：多人同时维护一个 wiki
- **回滚**：AI 改坏了的话，一键回滚

操作非常简单：

```
cd <你的 Vault>
git init
git add .
git commit -m "initial wiki"
```

配合 Obsidian 的 `obsidian-git` 插件，可以自动定期 commit，懒人福音。

12.4 用 Marp 自动出 PPT

Marp 是一种基于 Markdown 的 PPT 格式。Obsidian 有 Marp 插件。

接入之后，你可以让 AI 直接用 wiki 里的内容生成 PPT：

“基于 `wiki/analyses/xxx.md` 的内容，生成一份 8 页的 Marp 幻灯片，要求每页有标题、要点、配图建议。

直接导出 PPT，省去手动做幻灯片的时间。

12.5 用 Dataview 做动态看板

Dataview 是 Obsidian 的一个社区插件，可以基于 YAML frontmatter 自动生成动态表格。

举例：你的每个 source 页面 frontmatter 里都有 `created` 字段。让 Dataview 生成一张「最近一周新增的所有 source」的表格：

```
```dataview
TABLE created
FROM "wiki/sources"
WHERE created >= date(today) - dur(7 days)
SORT created DESC
```
```

每次刷新自动更新。把这种 Dataview 块放到 index.md 顶部，你的目录就是活的了。

第13章 典型应用场景

LLM Wiki 这套方法论几乎适合任何「需要长期沉淀知识」的场景。

13.1 个人

追踪自己的目标、健康、心理、自我提升。

- 把日记、文章、播客笔记、心理咨询记录全部丢进 raw
- AI 自动建出关于「你这个人」的实体页（你的目标、习惯、价值观）
- 定期 lint，看自己有哪些主张和半年前发生了变化

用它来「认识你自己」。

13.2 研究

某个主题深耕几周几月。

- 论文、行业报告、专家访谈、播客转录稿全部丢进 raw
- AI 帮你梳理出整个领域的实体（关键人物、机构、产品）、核心概念、争议焦点
- 你的研究 thesis 也是一份 analysis 页，会随着读得越多自动更新

适合做硕博论文、行业报告、深度调研。

13.3 读书

边读边记，到读完时你已经有了一本属于你自己的「读书伴侣 wiki」。

- 每读一章把笔记丢进 raw
- AI 自动建出角色页、地点页、概念页、情节线
- 跨章节的伏笔和回响会被自动发现
- 想想 Tolkien Gateway 那种粉丝维护的《指环王》wiki——只不过这次是你一个人的

特别适合读小说、读厚书、读经典。

13.4 公司 / 团队

把团队的 Slack 记录、会议纪要、客户访谈、项目文档全部喂进去。

- AI 自动维护一份「活的」内部 wiki
- 谁负责什么、什么时候做了什么决定、客户的痛点是什么，全都有迹可循
- 新人入职翻 wiki 就能上手
- 离职员工带走的知识，wiki 留住了

特别适合知识密集型团队，比如研究机构、咨询公司、产品团队。

13.5 其他

- 竞品分析
- 尽职调查 (DD)
- 旅行规划
- 课程笔记
- 爱好深耕（你喜欢的乐队、你养的植物、你研究的酒）

任何「值得长期投入精力的领域」，都值得有一个 LLM Wiki。

第14章 收尾

回顾整个系统，核心其实就三件事：

1. **写好 CLAUDE.md**，把规则定清楚。AI 是个非常听话的员工，但前提是你得先把员工手册写明白。
2. **把你收集的原始资料统统扔进 raw**。用 Web Clipper、用拖拽、用任何方式，把信息聚集到一个地方。
3. **舒舒服服地看着本地 AI 帮你把知识编译进 wiki**。剩下的，让 LLM 干。

这套方法论最厉害的地方，是它解决了一个 **Memex 七十多年来悬而未决的问题**：

到底由谁来做维护？

Vannevar Bush 在 1945 年描绘了 Memex 的设想——一个私人的、被精心维护的知识库。这个设想之所以从未真正实现，**不是因为技术不够（早就够了），而是因为没人愿意做那个维护的人。**

更新交叉引用、保持摘要新鲜、记录新数据推翻旧主张、维持几十个页面之间的一致性。

这些活又重要又枯燥，人类做不下来。

LLM 不会觉得无聊。LLM 不会忘记更新交叉引用。LLM 可以一次改 15 个文件。

于是 Memex 终于落地了。

你的知识库不再因为维护成本太高而被你放弃。它会持续变厚、变深、变独特——直到有一天，**它变成只有你才有的、专属于你的智能。**

这才是 LLM 时代，普通人最应该利用的红利。

“ 文档版作者：阿众（全平台账号 @我是阿众）

配套视频：B 站 / Bilibili

如果这份文档对你有用，欢迎转发给身边在折腾 AI 工具的朋友。